React + Full-Stack Blog Cheat Sheet

Learn React, and see how our whole system fits together

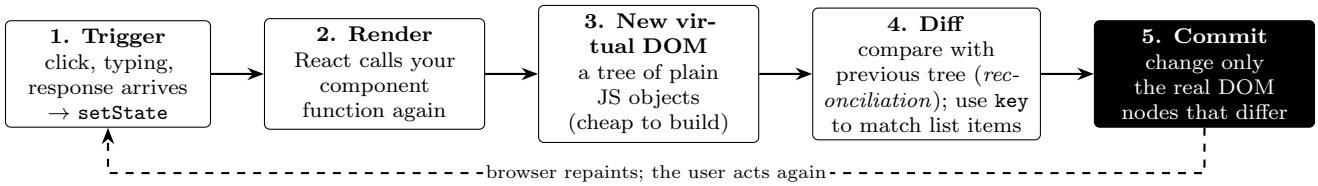
ELE-IWS-521 | React frontend · Express API · PostgreSQL on Supabase

How to use this sheet. Part A teaches the React ideas you will be asked about (components, props, state, the virtual DOM, hooks, routing). Part B draws our project: architecture, request flow, login flow, database, component tree. Part C is quick reference (endpoints, SQL, Git). Every example here comes from our own code, so open the file named in the text and read it next to the sheet.

Part A — React in one sitting

A1. The one big idea: UI = function of state

You do **not** tell the browser how to change the page. You describe what the page should look like for the current data (*state*); React works out the changes. When state changes React does three things: **render** (call your component again), **diff** (compare the new description with the old one), **commit** (touch only the real DOM nodes that differ).



A2. JSX is just function calls

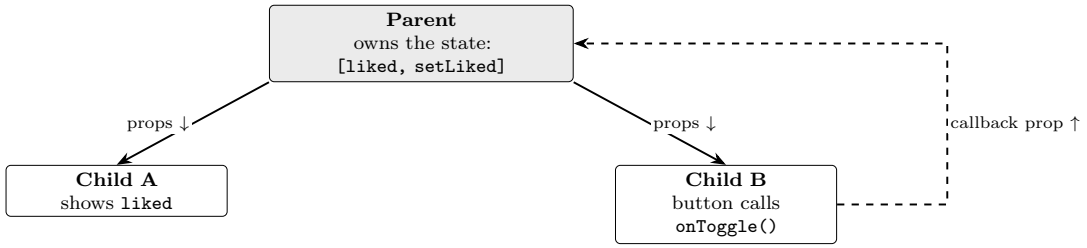
```
// What you write (JSX)
<button className="btn" onClick={save}>
  Save {count}
</button>
```

```
// What it becomes
jsx("button", {
  className: "btn", onClick: save,
  children: ["Save ", count],
})
// which returns a plain object:
// { type: "button", props: {...} }
```

Rules: one root element (or `<>...</>`); `className` not `class`; { } embeds any JavaScript *expression*; every tag must close (`<img />`). Components start with a **capital letter** so React can tell `<Navbar />` from `<nav>`.

A3. Components, props, and one-way data flow

A component is a function that takes **props** (inputs, read-only) and returns JSX. Data flows **down** through props; events flow **up** by calling a function the parent passed in. That makes bugs easy to trace: to find who changed the data, look *up* the tree.




```
function PostCard({ post, onDelete }) {          // props arrive as one object; we destructure it
  return (
    <article>
      <h2>{post.title}</h2>
      {post.status === "draft" && <span>Draft</span>}      {/* conditional rendering */}
      <button onClick={() => onDelete(post.id)}>Delete</button>   {/* event goes UP via a callback */}
    </article>
  );
}
{posts.map((p) => <PostCard key={p.id} post={p} onDelete={remove} />)}   // key = stable id, NOT the array index
```

A4. State: a component's memory

```
const [count, setCount] = useState(0);

setCount(count + 1);            // ok
setCount((c) => c + 1);         // best when the new
                                // value depends on the
     old
// NEVER mutate state:
items.push(x); setItems(items); // wrong
setItems([...items, x]);        // right (new array)
setUser({ ...user, name: "Ann" });   // new object
```

- State is a **snapshot**: inside one render, `count` never changes, even after `setCount`.
- Calling `setState` **schedules** a re-render; it does not change the variable you already hold.
- **Controlled input**: `value` comes from state, `onChange` writes back to state. React is the single source of truth.
- Keep state **as low as possible**; lift it up only to the nearest parent that needs to share it.
- Don't store what you can compute (e.g. `totalPages` from `total` and `limit`).

A5. Hooks

Hooks are functions starting with `use` that let a component remember things and react to the outside world. **Rules:** call them at the top level of a component (never inside `if`, loops or callbacks), and only from components or other hooks.

Hook	Use it for (and where we use it)
`useState`	Local values that change: form fields, open/closed, loaded data (Login form, Editor).
`useEffect`	Sync with something outside React: fetch data, timers, event listeners. Runs *after* render; re-runs when a dependency changes; the returned function is cleanup (`useFetch`, `useDebounce`).
`useContext`	Read shared data without passing props through every level (`useAuth()` reads `AuthContext`: user, login, logout).
`useRef`	A box that survives renders and does *not* trigger one: DOM nodes, timers, flags (`busy` in `LikeButton`).
`useMemo / useCallback`	Cache a calculation or a function between renders. Only when measured slowness or a memoised child needs it (`MarkdownView` memoises sanitised HTML).
`useReducer`	State with many related transitions; like `useState` with named actions. We did not need it.

Dependency array: `[]` run once after mount; `[a,b]` run when `a` or `b` change; no array = after every render (usually a bug).

Our own custom hook (`hooks/useDebounce.js`) — a custom hook is just a function that uses other hooks:

```
export default function useDebounce(value, delay = 400) {
  const [debounced, setDebounced] = useState(value);       // 1. remember a delayed copy of the value
  useEffect(() => {
    const timer = setTimeout(() => setDebounced(value), delay);   // 2. after ‘delay‘ ms with no change, copy it
    return () => clearTimeout(timer);       // 3. cleanup: value changed again, cancel the old timer
  }, [value, delay]);                       // 4. restart whenever value or delay changes
  return debounced;                         // 5. the search box calls the API with this, not every keystroke
}
```

A6. Fetching data with effects

```
useEffect(() => {
  let cancelled = false;                    // ignore a late response if we already moved on (stale update bug)
  setLoading(true);
  api.get(‘/posts?page=${page}‘)
    .then((data) => { if (!cancelled) { setData(data); setLoading(false); } })
    .catch((err)  => { if (!cancelled) { setError(err); setLoading(false); } });
  return () => { cancelled = true; };       // cleanup
}, [page]);                                  // re-fetch when the page changes
```

Every data screen needs **three states**: loading, error (with a retry), and empty. We wrapped this pattern in `useFetch`. Don't use an effect for things you can calculate during render or do inside an event handler (*You Might Not Need an Effect*).

A7. Routing, and client-side vs server-side rendering

```
<Routes>
  <Route element={<Layout />}>   {/* Navbar + <Outlet/>
     */}
    <Route index element={<Feed />} />
    <Route path="post/:id" element={<PostPage />} />
    <Route element={<ProtectedRoute roles={["admin"]}
     />}>
      <Route path="admin" element={<Admin />} />
    </Route>
    <Route path="*" element={<NotFound />} />
```

```
  </Route>
</Routes>

const { id } = useParams();            // /post/7 -> "7"
const navigate = useNavigate();        // navigate("/login
    ")
const [params, setParams] = useSearchParams();  // ?q=
    maize
```

- `<Link>` / `<NavLink>` change the URL *without* a page reload; plain `<a>` reloads.
- `<Outlet />` is where the matching child route renders.
- **URL as state:** we keep `page`, `q`, `tag` in the URL so links can be shared and Back works.
- `ProtectedRoute` redirects visitors who are not allowed. It is a *convenience*: the API still checks every request.
- `React.lazy` + `Suspense` download rarely used pages only when opened (saves data).

	Client-side rendering (our app)	**Server-side rendering (e.g. Next.js)**
First load	Browser gets a near-empty HTML file, then downloads JS, then fetches data and draws.	Server sends finished HTML; JS then "hydrates" it.
Search engines / link previews	May see an empty page unless pre-rendered.	See full content.
After first load	Very fast navigation, only JSON travels.	Also fast, plus extra server work.
Hosting	Static files (Vercel) + separate API.	Needs a server runtime.

A8. Where should state live? (our choices)

Kind of state	**Where**	**Why**
Form fields, open/closed	`useState` in the component	Only that component cares.
Logged-in user, token	`AuthContext` (Context API)	Many distant components need it; changes rarely.
Page, search text, tag	URL (`useSearchParams`)	Shareable, survives refresh, Back button works.
Server data (posts, comments)	`useFetch` + local state	Small app; a cache library (TanStack Query) or Redux would be extra weight.
Unsaved draft	`localStorage` via `useAutosaveDraft`	Must survive refresh or a dropped connection.

A9. Optimistic update (likes and comments)

Show the result *now*, confirm with the server *after*, undo if it failed. Source: `components/LikeButton.jsx`.

```
const previous = like;                                  // 1. remember how things were
setLike({ liked: !previous.liked, count: previous.count + (previous.liked ? -1 : 1) });   // 2. update the UI
    immediately
try {
  const res = previous.liked ? await api.del(url) : await api.post(url);
  setLike({ liked: res.liked, count: res.likeCount });   // 3. the server's answer is the truth
} catch {
  setLike(previous);                                    // 4. rollback
  setError("Could not save your like.");                // 5. tell the user
}
```

A10. Bugs you will meet (and the fix)

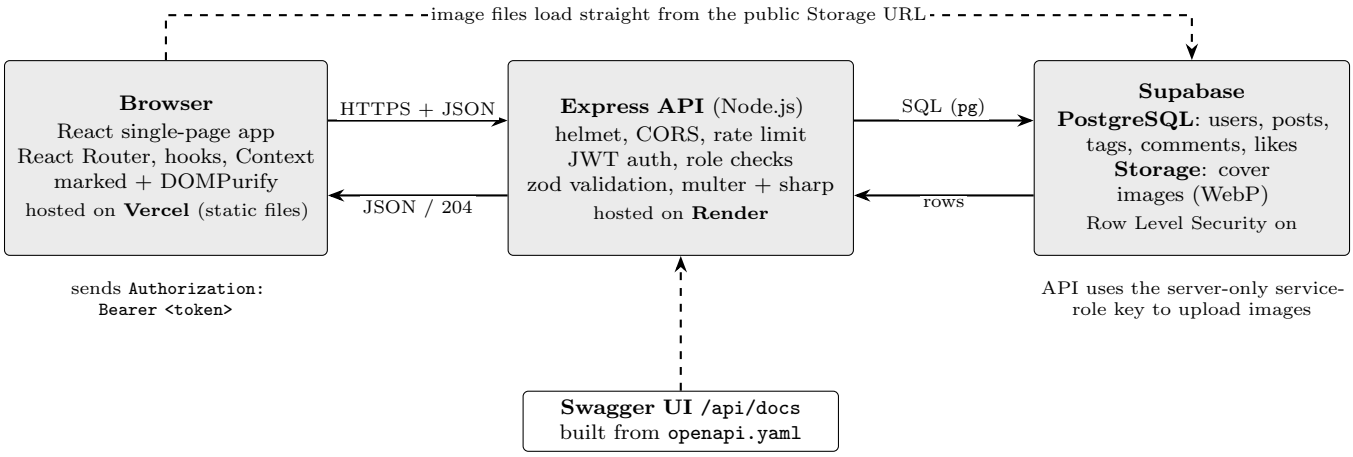
Symptom	Usual cause → fix
Warning: each child needs a unique `key`	Missing `key` in `.map()` → use the item's database id.
Screen doesn't update after `setState`	You mutated the old array/object → create a new one (`[...a, x]`, `{...o, k: v}`).
Infinite loop of requests	`useEffect` sets state it also depends on, or has no dependency array → fix the array.
Old value inside a callback (stale closure)	Closure captured an old render → use the functional update `setX(x => ...)` or add the dependency.
Shows `[object Object]` or crashes	Rendering an object → render its fields.
Data arrives for a page you already left	Missing cleanup → the `cancelled` flag or `AbortController`.
Typing in an input does nothing	Controlled input without `onChange` → add `onChange` that sets state.
Works in dev, 404 on refresh in production	SPA needs a rewrite to `index.html` → `vercel.json`.

A11. Two-week study path

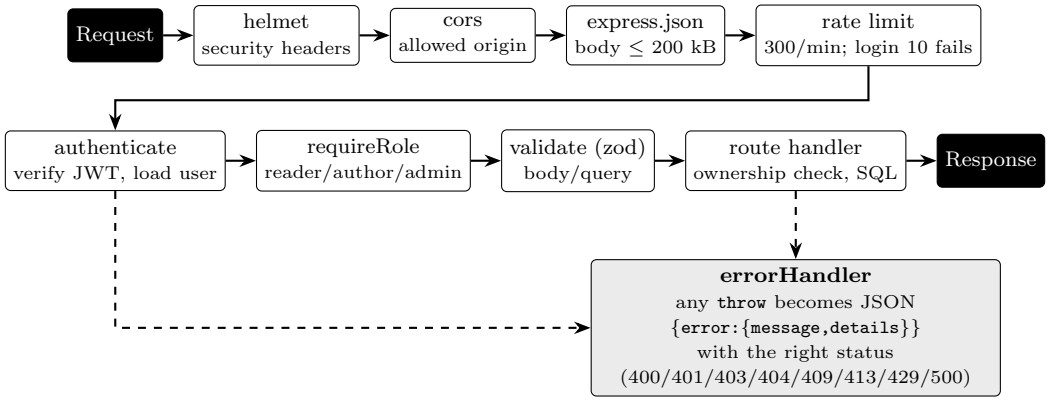
Days 1–3	react.dev *Learn*: Describing the UI (JSX, props, conditional rendering, lists, keeping components pure).
Days 4–6	Adding Interactivity: events, state, *Render and Commit*, state as a snapshot, updating objects and arrays.
Days 7–9	Managing State and Escape Hatches: sharing state, *Preserving and Resetting State* (keys), Context, refs, effects.
Days 10–12	*Reusing Logic with Custom Hooks*. Read our hooks and components; change one thing per file and watch what breaks.
Days 13–14	Rehearse: explain one component and one hook out loud, line by line, without notes.

Part B — Our project's architecture

B1. System architecture

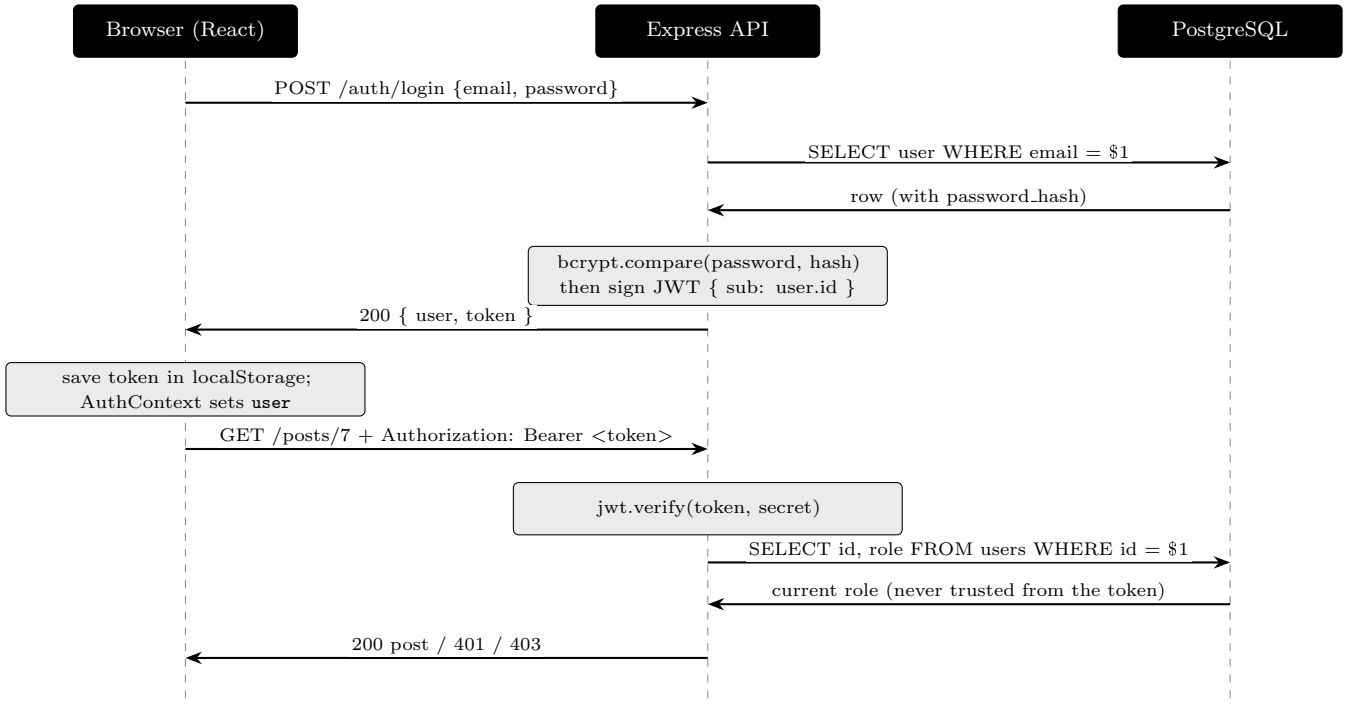


B2. What happens to one API request (middleware pipeline)



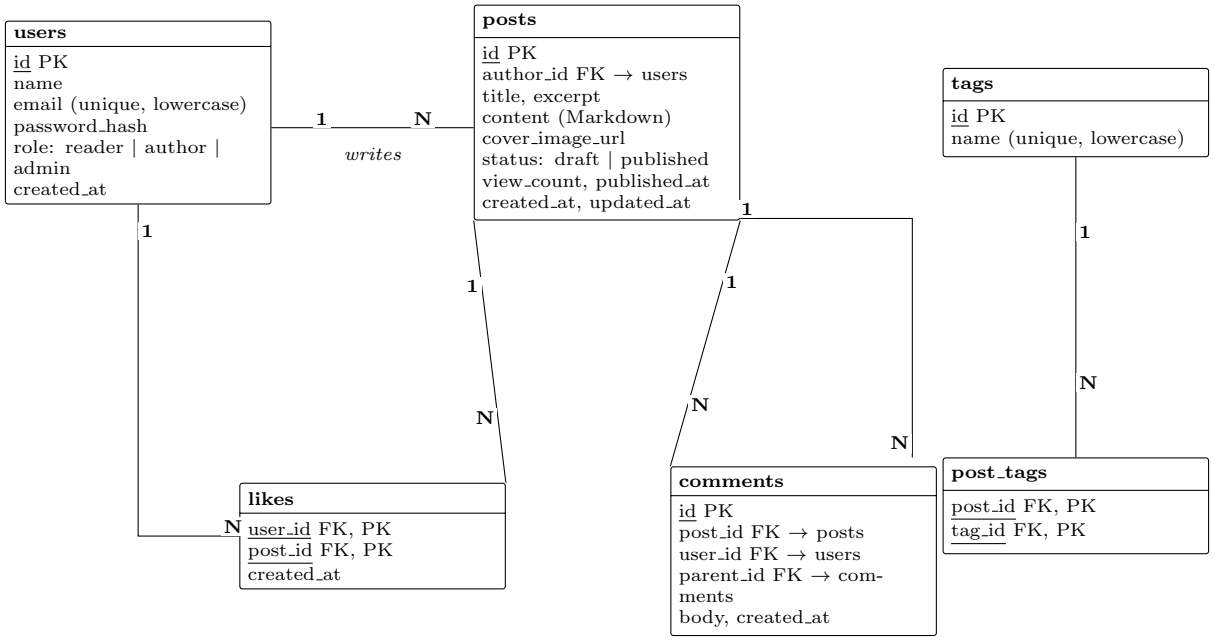
Failing a step stops the chain: no token → **401**; wrong role or not the owner → **403**; bad input → **400** with field messages; missing/hidden resource → **404**.

B3. Login, token, and a protected request



Logout = the browser forgets the token (JWTs are stateless). **Passwords**: only the bcrypt hash is stored. **Same error** for "unknown email" and "wrong password" so attackers cannot discover which emails exist.

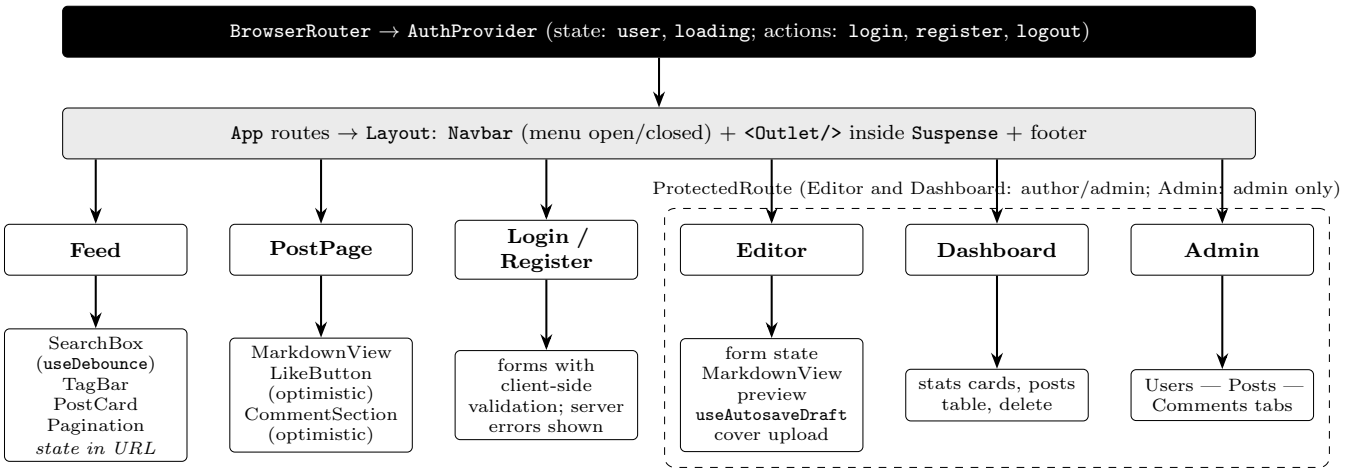
B4. Database (ER diagram)



Not drawn, to keep it readable: `comments.user_id` also references `users` (who wrote it), and `comments.parent_id` points back to another row of `comments` (replies, one level only).

Constraints doing real work: `likes` primary key $(user_id, post_id)$ = one like per user per post; a `BEFORE INSERT` trigger on `comments` rejects a reply to a reply or a reply on a different post; `ON DELETE CASCADE` removes a post's comments, likes and tags with it; `CHECK` constraints limit lengths and force lowercase emails; Row Level Security is enabled with no policies, so Supabase's public REST API cannot read the tables (our server connects as the database owner, which bypasses it).

B5. React component tree and where state lives



B6. Who can do what

Action	Anon	Reader	Author	Admin
Read published posts, search, filter by tag	•	•	•	•
Like, comment, reply; delete own comment		•	•	•
Create posts, upload cover, dashboard			•	•
Edit / delete *own* post			•	•
Edit / delete *any* post, delete *any* comment, see others' drafts, manage roles				•

Enforced on the server (`requireRole` and ownership checks); hiding buttons in React is only for convenience.

B7. API endpoints (base `/api`; full examples in `openapi.yaml`)

Method	Path	Purpose / access
GET	/health	Health check (pings database)
POST	/auth/register	Create reader/author; returns token
POST	/auth/login	Log in; rate limited
GET	/auth/me	Current user (token)
GET	/posts	Feed: `page, limit, q, tag, author`
GET	/posts/:id	One post (drafts: owner/admin); counts a view
POST	/posts	Create (author, admin)
PUT	/posts/:id	Replace (owner, admin)
DELETE	/posts/:id	Delete (owner, admin)
POST, DELETE	/posts/:id/like	Like / unlike (any logged-in user)
GET, POST	/posts/:id/comments	List (nested replies) / add or reply
DELETE	/comments/:id	Owner or admin
GET	/tags	Tags with published posts
GET	/dashboard/posts	My posts with views, likes, comments (author, admin)
GET	/admin/users	Admin: users
PATCH	/admin/users/:id/role	Admin: change role (not your own)
GET	/admin/posts, /admin/comments	Admin: moderation lists
POST	/uploads/cover	Image upload → resized 1200 px WebP

B8. Status codes we use

200 OK 201 created 204 deleted (no body) 400 validation failed 401 not logged in / bad token 403 logged in but not allowed 404 not found (also hides other people's drafts) 409 duplicate (email) 413 file too large 429 too many login attempts 500 our bug.

Part C — Quick reference

C1. The SQL that matters (from our routes)

```
-- Feed: filter + newest first + one page (all values are $1, $2... parameters, never string-glued: no SQL injection)
SELECT p.id, p.title, json_build_object('id', u.id, 'name', u.name) AS author,
       (SELECT count(*)::int FROM likes l WHERE l.post_id = p.id) AS "likeCount"
FROM posts p JOIN users u ON u.id = p.author_id
WHERE p.status = 'published' AND (p.title ILIKE $1 OR p.content ILIKE $1)
ORDER BY p.published_at DESC, p.id DESC
LIMIT $2 OFFSET $3;                          -- OFFSET = (page - 1) * limit

-- Like once: the primary key (user_id, post_id) makes a duplicate impossible; DO NOTHING makes the API idempotent
INSERT INTO likes (user_id, post_id) VALUES ($1, $2) ON CONFLICT DO NOTHING;

-- Tags for a post: create missing tags, then link them (inside a transaction with the post insert)
INSERT INTO tags (name) SELECT unnest($1::text[]) ON CONFLICT (name) DO NOTHING;
INSERT INTO post_tags (post_id, tag_id) SELECT $1, id FROM tags WHERE name = ANY($2::text[]);

-- Dashboard totals per post
SELECT p.id, p.view_count AS views,
       (SELECT count(*)::int FROM likes l WHERE l.post_id = p.id)    AS likes,
       (SELECT count(*)::int FROM comments c WHERE c.post_id = p.id) AS comments
FROM posts p WHERE p.author_id = $1;
```

Transaction = all statements succeed or none do (`BEGIN` … `COMMIT`, `ROLLBACK` on error): creating a post and its tags must not leave a post without tags.

C2. Environment variables and deployment

Variable	Where	Meaning
DATABASE_URL	Render	Supabase *session pooler* string (IPv4-friendly; the direct string is IPv6-only)
DATABASE_SSL	Render	`true` for Supabase
JWT_SECRET	Render	32+ random characters; never in Git
CLIENT_ORIGIN	Render	Your Vercel URL (CORS)
SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY, SUPABASE_BUCKET	Render	Image uploads (service-role key = server only)
VITE_API_URL	Vercel	`https://<render-app>.onrender.com/api`

Order: Supabase (run `schema.sql`, create public bucket `covers`) → Render API → Vercel client → seed once → change demo passwords. Free Render servers sleep: open the site 10 minutes before the demo.

C3. Git habits that earn marks

At least **5 commits per member**, small and described by what changed: `add ownership check to PUT /posts/:id`, not `update`. One branch per feature (`feat/comments`), merge by pull request, never commit `.env` or keys. Tag the final commit: `git tag final-submission && git push --tags`.

C4. References

- React: `https://react.dev/learn` (Render and Commit: `https://react.dev/learn/render-and-commit`; Preserving and Resetting State: `https://react.dev/learn/preserving-and-resetting-state`; Custom Hooks: `https://react.dev/learn/reusing-logic-with-custom-hooks`; Context: `https://react.dev/learn/passing-data-deeply-with-context`)
- React Router: `https://reactrouter.com` Vite: `https://vite.dev`
- Express: `https://expressjs.com` JWT (RFC 7519): `https://datatracker.ietf.org/doc/html/rfc7519`
- OWASP Password Storage: `https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html`
- OWASP XSS Prevention: `https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html`
- PostgreSQL: `https://www.postgresql.org/docs/current/` Supabase connect: `https://supabase.com/docs/guides/database/connecting-to-postgres`
- Supabase RLS: `https://supabase.com/docs/guides/database/postgres/row-level-security` Storage: `https://supabase.com/docs/guides/storage`
- MDN Fetch: `https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API` Render: `https://render.com/docs` Vercel: `https://vercel.com/docs`